

```

"""
Renders a list of Line objects (from
geometry.py) to an SVG dieline.

Convention (matches real packaging CAD,
e.g. Esko ArtiosCAD):
    - CUT lines -> solid black stroke (die
    actually cuts through the board)
    - CREASE lines -> dashed stroke (die
    scores but does not cut; fold line)
"""

from geometry import Line

def lines_to_svg(lines: list[Line], margin:
float = 10.0) -> str:
    """Build the SVG markup as a string
    (used for inline rendering + downloads)."""
    xs = [p for ln in lines for p in
(ln.x1, ln.x2)]
    ys = [p for ln in lines for p in
(ln.y1, ln.y2)]
    min_x, max_x = min(xs), max(xs)
    min_y, max_y = min(ys), max(ys)

    width = (max_x - min_x) + 2 * margin
    height = (max_y - min_y) + 2 * margin

    def tx(x: float) -> float:

```

```

        return x - min_x + margin

    def ty(y: float) -> float:
        # flip y so the pattern reads top-
        to-bottom like a real layout
        return (max_y - y) + margin

    svg_lines = [
        f'<svg
xmlns="http://www.w3.org/2000/svg" '
        f'viewBox="0 0 {width:.2f}
{height:.2f}" width="{width:.2f}mm"
height="{height:.2f}mm">',
        f'<rect x="0" y="0" width="
{width:.2f}" height="{height:.2f}"
fill="white"/>',
    ]

    for ln in lines:
        stroke = "black"
        dash = "" if ln.kind == "cut" else
'stroke-dasharray="3,2"'
        width_attr = "0.5" if ln.kind ==
"cut" else "0.35"
        svg_lines.append(
            f'<line x1="{tx(ln.x1):.2f}"
y1="{ty(ln.y1):.2f}" '
            f'x2="{tx(ln.x2):.2f}" y2="
{ty(ln.y2):.2f}" '
            f'stroke="{stroke}" stroke-
width="{width_attr}" {dash}/>'
        )

    svg_lines.append("</svg>")

```

```
return "\n".join(svg_lines)
```

```
def export_svg(lines: list[Line], path:
str, margin: float = 10.0) -> None:
    """Write the SVG to a file (used by the
    CLI / test script)."""
    with open(path, "w") as f:
        f.write(lines_to_svg(lines,
margin))
```